

同济大学计算机系

计算机组成原理课程实验报告



学 号	<u>2351520</u>
姓 名	<u>毛星博</u>
专 业	<u>计算机科学与技术</u>
授课老师	<u>陈永生</u>

CPU 中断实验

一、实验内容

除了通常的运算功能之外，任何处理器都需要一些部件来处理中断、配置选项以及需要某种机制来监控诸如片上高速缓存和定时器等功能。在 MIPS CPU 中，异常或者中断发生时的行为以及怎样处理都是由 CP0 控制寄存器和几条特殊指令来定义和控制的。

本实验不需要实现 CP0 的全部功能，只关注 CP0 的寄存器读写和对中断异常的处理部分。通过本次实验，了解 CP0 异常处理的实现原理，学习如何实现 CP0 对异常处理的处理，并使用 Verilog 硬件描述语言实现一个简化的 CP0 模块。

实现 6 条指令：

mfc0 / mtc0: 实现 CPU 与 CP0 寄存器之间的数据读写。

syscall / break / teq: 实现这三条指令触发异常的逻辑。

eret: 实现从异常处理程序返回的指令，该指令会恢复 Status 寄存器状态，并使 CPU 跳转回 EPC 保存的地址。

关键 CP0 寄存器：

Status (12 号): 处理器状态和控制寄存器。实验中主要使用其 IE 位 (bit 0) 来全局控制中断，并使用特定位 (bit 8-10) 作为 syscall、break 和 teq 指令的屏蔽位。

Cause (13 号): 记录并保存上一次发生异常的原因。实验主要使用其 ExcCode 字段 (bits 6:2) 来区分不同的异常类型，例如 syscall 为 1000，break 为 1001。

EPC (14 号): 异常程序计数器，用于保存在发生异常时当前指令的地址，以便在处理完成后能够返回。

异常处理流程：

CPU 执行到 syscall、break 或 teq 等指令时，会向 CP0 发出异常请求。

CP0 检查 Status 寄存器，确定中断没有被全局关闭或被特定屏蔽。

若可以响应，CP0 会将当前指令的地址 pc 保存到 EPC 寄存器，并将异常类型码写入 Cause 寄存器。

CP0 会关闭中断（修改 Status 寄存器）以防止嵌套异常，并向 CPU 提供统一的异常处理入口地址：0x4。

二、硬件逻辑图：无

（实验步骤中要求用 logisim 画图的实验，在该部分给出 logisim 原理图，否则该部分在实验报告中不用写）

三、模块建模

CP0 模块

寄存器访问与管理

该模块内部实现了三个核心的控制寄存器：Status（状态寄存器）、Cause（原因寄存器）和 EPC（异常程序计数器）。

它提供 mfc0 和 mtc0 指令接口，允许 CPU 将这些控制寄存器的值读入通用寄存器，

或将通用寄存器的值写入控制寄存器，从而实现对处理器状态的软件配置与查询。

异常处理与响应

当 CPU 检测到 `syscall`、`break` 或 `teq` 等异常指令时，CP0 模块会接收到异常请求信号。

模块会检查 `Status` 寄存器中的全局中断使能位（`IE`）和特定异常的屏蔽位，以决定是否响应异常。

若异常被响应，模块会自动执行一系列硬件操作：将当前指令的程序计数器（`PC`）存入 `EPC` 寄存器作为返回地址，将异常类型码写入 `Cause` 寄存器，并更新 `Status` 寄存器以禁止后续中断。

同时，模块向 CPU 输出一个固定的异常处理入口地址（`exc_addr`），引导程序跳转。

异常返回

响应 `eret` 指令时，CP0 模块负责恢复 `Status` 寄存器的状态，重新使能中断。CPU 硬件则配合此操作，将 `EPC` 寄存器中保存的地址加载回程序计数器 `PC`，从而使程序从异常中返回，继续执行被打断前的任务。

CP0 模块代码：

```
`timescale 1ns / 1ps
module CP0(
    input        clk,
    input        rst,
    // 与 CPU 指令交互的接口
    input        mfc0,      // CPU 指令 Mfc0 (读 CP0)
    input        mtc0,      // CPU 指令 Mtc0 (写 CP0)
    input        eret,      // CPU 指令 ERET (异常返回)
    // 异常处理相关接口
    input        exception,  // 来自 CPU 的异常请求信号
    input [4:0]   cause,     // 来自 CPU 的异常类型码 (ExcCode)
    input        intr,      // 外部中断请求 (本次代码未处理)
    // 数据与地址总线
    input [31:0]  pc,        // 当前指令的地址
    input [4:0]   Rd,        // 指定 CP0 寄存器的地址
    input [31:0]  wdata,     // 从通用寄存器写入 CP0 的数据
    // 输出信号
    output [31:0] rdata,     // 从 CP0 读出到通用寄存器的数据
    output [31:0] status,    // 输出 Status 寄存器的当前值
    output reg    timer_int, // 定时器中断信号 (本次实验未选择)
    output [31:0] exc_addr   // 异常入口地址
);

reg [31:0] status_reg; // 12 号寄存器：状态控制
reg [31:0] cause_reg;  // 13 号寄存器：保存异常原因
reg [31:0] epc_reg;    // 14 号寄存器：保存异常返回地址
reg masked;           // 是否被屏蔽

assign status = status_reg;
```

```

assign exc_addr = 32'h0000_0004;

assign rdata = (mfc0 && Rd == 12) ? status_reg :
               (mfc0 && Rd == 13) ? cause_reg :
               (mfc0 && Rd == 14) ? epc_reg :
               32'b0;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        // 复位时, 清空寄存器
        status_reg <= 32'h0;
        cause_reg  <= 32'h0;
        epc_reg    <= 32'h0;
        timer_int  <= 1'b0;
        //优先级 exception > mtc0 > eret
    end
    else if (exception && status_reg[0]) begin // 检查是否有异常请求且中断未被屏蔽
        masked = 1'b0;

        // 检查特定异常是否被屏蔽
        case (cause)
            5'd8: masked = status_reg[8]; // syscall
            5'd9: masked = status_reg[9]; // break
            5'd13: masked = status_reg[10]; // teq
            default: masked = 1'b1;
        endcase
        if (!masked) begin
            // 异常未被屏蔽, 开始处理
            epc_reg <= pc; // 保存返回地址
            cause_reg[6:2] <= cause; // 设置 Cause 寄存器的 ExcCode 字段
            status_reg <= status_reg << 5; //status 寄存器左移 5 位来保存并关闭中断
        end
    end else if (eret) begin
        status_reg <= status_reg >> 5; //右移 5 位以恢复进入异常前的 Status 状态
    end else if (mtc0) begin
        // 当 mtc0 指令有效时, 根据 Rd 指定的地址写入数据
        case (Rd)
            12: status_reg <= wdata;
            13: cause_reg  <= wdata;
            14: epc_reg    <= wdata;
            default: ;
        endcase
    end
end
endmodule

```

四、测试模块建模

```
`timescale 1ns / 1ps
module CPO_tb;

    // 信号声明
    reg      clk;
    reg      rst;
    reg      mfc0;
    reg      mtc0;
    reg      eret;
    reg      exception;
    reg [4:0] cause;
    reg      intr;
    reg [31:0] pc;
    reg [4:0] Rd;
    reg [31:0] wdata;

    wire [31:0] rdata;
    wire [31:0] status;
    wire      timer_int;
    wire [31:0] exc_addr;

    // 实例化被测模块
    CPO dut (
        .clk(clk),
        .rst(rst),
        .mfc0(mfc0),
        .mtc0(mtc0),
        .eret(eret),
        .exception(exception),
        .cause(cause),
        .intr(intr),
        .pc(pc),
        .Rd(Rd),
        .wdata(wdata),
        .rdata(rdata),
        .status(status),
        .timer_int(timer_int),
        .exc_addr(exc_addr)
    );

    // 时钟生成
    initial begin
```

```

        clk = 0;
    end
    always #5 clk = ~clk;

// 测试序列
initial begin
    // 初始化和复位
    // rst 为高电平期间，所有输出和内部寄存器应被复位为 0。
    rst = 1;
    mfc0 = 0; mtc0 = 0; eret = 0; exception = 0;
    cause = 5'd0; intr = 0; pc = 32'h0; Rd = 5'd0; wdata = 32'h0;

    #20;
    rst = 0;
    #10;

    // 测试 mtc0 (写) 和 mfc0 (读)
    // 向 Status 寄存器写入 0x1 (开中断)
    // 下一个时钟上升沿后，status 信号应变为 32'h1。
    mtc0 = 1;
    Rd = 12; // Status 寄存器地址
    wdata = 32'h0000_0001;
    @(posedge clk);
    mtc0 = 0;
    #5;

    // 读 Status 寄存器 (mfc0)
    // mfc0 为高电平期间，rdata 信号的值应等于 status 的值 (32'h1)。
    mfc0 = 1;
    Rd = 12;
    #10;
    mfc0 = 0;
    #10;

    // 测试异常处理 (syscall)
    // 下一个时钟上升沿后，epc_reg 变为 PC 值，
    // cause_reg 的 ExcCode 字段变为 8，status 信号变为 32'h20。
    exception = 1;
    cause = 5'd8; // Syscall 的异常码
    pc = 32'h0040_0020;
    @(posedge clk);
    exception = 0;
    pc = 32'h0;
    #10;

```

```

// 测试异常返回 (eret)
// 在波形图上观察：下一个时钟上升沿后，status 信号恢复为 32'h1。
eret = 1;
@(posedge clk);
eret = 0;
#10;

// 测试异常屏蔽
// 设置 Status 寄存器，使能中断(IE=1)，同时屏蔽 syscall (Mask=1)
// 下一个时钟上升沿后，status 信号变为 32'h101。
mtc0 = 1;
Rd = 12; // Status 寄存器
wdata = 32'h0000_0101; // bit8=1 (屏蔽 syscall), bit0=1 (开中断)
@(posedge clk);
mtc0 = 0;
#10;

// 再次尝试触发 syscall 异常
// 下一个时钟上升沿后，epc_reg, dut.cause_reg, status 信号均不发生变化
exception = 1;
cause = 5'd8;
pc = 32'h0040_0088;
@(posedge clk);
exception = 0;
#10;

#50;
end
endmodule

```

五、实验结果

modelsim 仿真波形图：

syscall 异常。从波形图上可以明确看到，尽管 exception 信号有效，但在时钟上升沿之后，epc_reg, cause_reg 和 status_reg 的值均未发生任何变化。证明了异常屏蔽逻辑工作正常，CP0 模块会检查屏蔽位并正确地忽略被屏蔽的中断请求。

综上所述，本次仿真波形图的结果与理论设计预期符合。CP0 模块在复位、寄存器访问、异常处理、异常返回和异常屏蔽等所有关键功能测试中均表现正确，验证了本次硬件设计的正确性和有效性。